

Универзитет Привредна Академија у Новом Саду
Факултет за економију и инжењерски менаџмент

Дипломски рад

**End-to-end аутономна навигација
возила: приступ дубоког учења
коришћењем конволуционих
неуронских мрежа**

Ментор:
Проф. др Драган Солеша

Студент:
Огњен Јовановић СИ 82/23

Нови Сад, јануар 2026.

Апстракт

Аутономна возила представљају значајан напредак у модерним транспортним системима. Конвенционална решења за аутономну вожњу обично се ослањају на модуларне pipeline-ове који обухватају подсистеме за перцепцију, планирање и управљање, што захтева комплексну интеграцију и обимно ручно пројектовање карактеристика. Новији напредак дубоког учења омогућио је end-to-end парадигме учења које директно мапирају сирове улазе на управљачке команде возила.

Овај дипломски рад представља дизајн и имплементацију система за аутономну навигацију засновану на визији, користећи behavioral cloning и конволуционе неуронске мреже (CNN), имплементираног на уградбену платформу возила базирану на Raspberry Pi 4. Предложени систем обрађује RGB слике камере резолуције 224x224 пиксела и користи CNN архитектуру изведену из PilotNet модела за предикцију управљачких команди у реалном времену. Како би се осигурала рачунарска изводљивост на хардверу са ограниченим ресурсима, примењена је динамичка квантизација модела ради смањења латенције инференције уз очување тачности предикције. Скуп података за тренинг приближно 20.000 парова слика-команда прикупљених кроз демонстрацију људске вожње.

Експериментална евалуација показује да тренирани модел постиже перформансе у реалном времену на платформи Raspberry Pi 4, омогућавајући аутономну навигацију у затвореним ходничким окружењима уз поуздано избегавање препрека. Квантизација модела даје убрзање инференције од приближно 1.2x уз занемарљиво смањење тачности. Штавише, интеграција ултразвучних сензора близине значајно побољшава безбедност и робусност система током рада у реалним условима.

Резултати демонстрирају да се end-to-end приступи учења могу ефикасно имплементирати на јефтиним уградбеним платформама, чиме се побољшава приступачност истраживања аутономне вожње за едукативне и експерименталне примене.

Кључне речи: *End-to-End Learning, Behavioral Cloning, Convolutional Neural Networks (CNN), Autonomous Navigation, Computer Vision, Embedded Systems, Real-Time Inference, Model Quantization, Raspberry Pi, Edge Computing.*

Садржај

Апстракт	2
1. Увод	6
1.1 Мотивација	6
1.2 Дефиниција проблема	6
1.3 Циљеви истраживања	6
1.4 Преглед система	7
1.5 Доприноси	8
1.6 Организација рада	8
2. Сродни радови	8
2.1 Приступи аутономној вожњи	8
2.1.1 Методе модуларног pipeline-a	8
2.1.2 End-to-End учења	9
2.2 Behavioral Cloning и имитационо учење	9
2.3 CNN архитектура за визуелну перцепцију	9
2.4 Уградбена вештачка интелигенција и Edge Computing	9
2.5 Сродне платформе	9
2.6 Истраживачки јаз	10
3. Архитектура и дизајн система	10
3.1 Архитектонски преглед	10
3.2 Хардверска архитектура	10
3.2.1 Уградбена рачунарска јединица	10
3.2.2 Актуаторски систем	11
3.2.3 Сензорски подсистем	12
3.2.5 Конфигурација GPIO пинова	14
3.3 Софтверска архитектура	15
3.3.1 Архитектонски преглед	15
3.3.2 Уградбени контролни слој	16
3.3.3 Слој перцепције и инференције	16
3.3.4 Безбедносни слој и руковање грешкама	17
3.3.5 Слој мобилне апликације	17
3.4 Комуникациона архитектура	18
3.4.1 Мрежна топологија	18
3.4.2 Комуникациони канали	18
3.5 Безбедносна архитектура	19
3.6 Образложење дизајна	20
4. Методологија учења и развој модела	21
4.1 Формулација проблема	21
4.2 Framework за Behavioral Cloning	21
4.3 Репрезентација улаза	22
4.4 Архитектура модела	22
4.4.1 Дизајнерска инспирација	22
4.4.2 Структура мреже	23
4.4.3 Динамичко рачунање величине карактеристика	24
4.5 Методологија прикупљања скупа података	24
4.5.1 Активизација демонстрација	24

4.5.2	Покривеност образаца вожње	24
4.5.3	Стратегијско филтрирање података	24
4.5.4	Састав скупа података	25
4.6	Аугментација података	25
4.6.1	Pipeline за аугментацију	25
4.7	Процедура тренинга	26
4.7.1	Функција губитка	26
4.7.2	Конфигурација оптимизације	26
4.7.3	Стратегија регуларизације	26
4.8	Оптимизација модела за уградбену примену	27
4.8.1	Динамичка квантизација	27
4.8.2	TorchScript компилација	27
5.	Детаљи имплементације	28
5.1	Преглед	28
5.2	Имплементација прикупљених података	28
5.2.1	Скрипта за прикупљање	28
5.2.2	Обрада контролних сигнала	28
5.3	Имплементација pipeline-а за тренинг	29
5.3.1	Dataset класа	29
5.3.2	Структура скрипте за тренинг	29
5.4	Имплементација уградбеног сервера	29
5.4.1	Језгро контролног сервера	29
5.4.2	Вишенитна архитектура	30
5.4.3	Извршење инференције	30
5.4.4	Стратегије контроле само воланом	30
5.5	Интеграција ултразвучних сензора	31
5.5.1	Мерење дистанце	31
5.5.2	Логика детекције препрека	31
5.5.3	Процедура хитног избегавања	31
5.6	Имплементација мобилне апликације	32
5.6.1	Framework и архитектура	32
5.6.2	Интерфејс за ручну контролу	32
5.6.3	HUD за визуализацију сензора	32
5.6.4	Обрада MJPEG stream-а	32
5.6.5	Мрежна робусност	33
5.7	Развојно окружење	33
6.	Експериментална евалуација	33
6.1	Преглед	33
6.2	Експериментална поставка	34
6.2.1	Хардверско окружење	34
6.2.2	Софтверско окружење	34
6.3	Анализа скупа података	34
6.3.1	Дистрибуција управљања воланом	34
6.3.2	Дистрибуција гаса	35
6.4	Перформансе тренинга	35
6.4.1	Понашање конвергенције	35

6.4.2 Коначне перформансе модела	35
6.5 Перформансе уградбене инференције	36
6.5.1 Латенција инференције	36
6.5.2 Искоришћење ресурса	36
6.6 Евалуација аутономне вожње	36
6.6.1 Методологија евалуације	36
6.6.2 Квалитативна запажања	36
6.6.3 Перформансе избегавања препрека	36
6.6.4 Случајеви неуспеха	37
6.7 Анализа генерализације	37
7. Ограничења и дизајнерски компромиси	37
7.1 Преглед	37
7.2 Ограничење скупа података	37
7.3.1 Ограничења монокуларне визије	37
7.3.2 Осетљивост на окружење	38
7.4 Ограничења модела	38
7.4.1 Одсуство темпоралног моделовања	38
7.4.2 Реактивна контрола	38
7.5 Компромиси уградбеног система	38
7.5.1 Рачунарска ограничења	38
7.5.2 Стратегија фиксног гаса	38
7.6 Компромиси безбедносне архитектуре	38
8. Закључак и будући рад	39
8.1 Закључак	39
8.2 Остварени циљеви	39
8.3 Будући рад	39
8.3.1 Темпорални модели учења	39
8.3.2 Побољшање генерисање скупа података	39
8.3.3 Фузија сензора	40
8.3.4 Адаптивна контрола брзине	40
8.3.5 Тренинг заснован на симулацији	40
8.4 Завршне напомене	40
9. Литература	41
10. Сумирани преглед слика	42
11. Прилог: фотографије vozila	43

1. Увод

1.1 Мотивација

Растућа релевантност аутономних возила покренула је обимна истраживања у академским и индустријским доменима. Упркос значајном напретку, комерцијални системи за замовожњу остају рачунарски интезивни и финансијски недоступни за већину образовних институција и малих истраживачких група.

Сходно томе, постоји снажна потражња за јефтиним експерименталним платформама које омогућавају практично истраживање принципа аутономне вожње. Новији развој edge вештачке интелигенције олакшава перцепцију и управљање у реалном времену директно на уградбеним уређајима, смањујући латенцију система и елиминишући зависност од cloud рачунарства.

1.2 Дефиниција проблема

Примарни изазов који се обрађује у овом раду је реализација аутономне навигације возила на хардверским платформама са ограниченим ресурсима уз одржавање поузданих перформанси у реалном времену.

Главна истраживачка питања формулисана су на следећи начин:

1. Да ли behavioral cloning може ефикасно научити управљање воланом из демонстрација људске вожње?
2. Како се конволуционе неуронске мреже могу оптимизовати за примену на уградбеном хардверу?
3. Који компромиси настају између комплексности модела и латенције инференције у окружењима са ограниченим ресурсима?

1.3 Циљеви истраживања

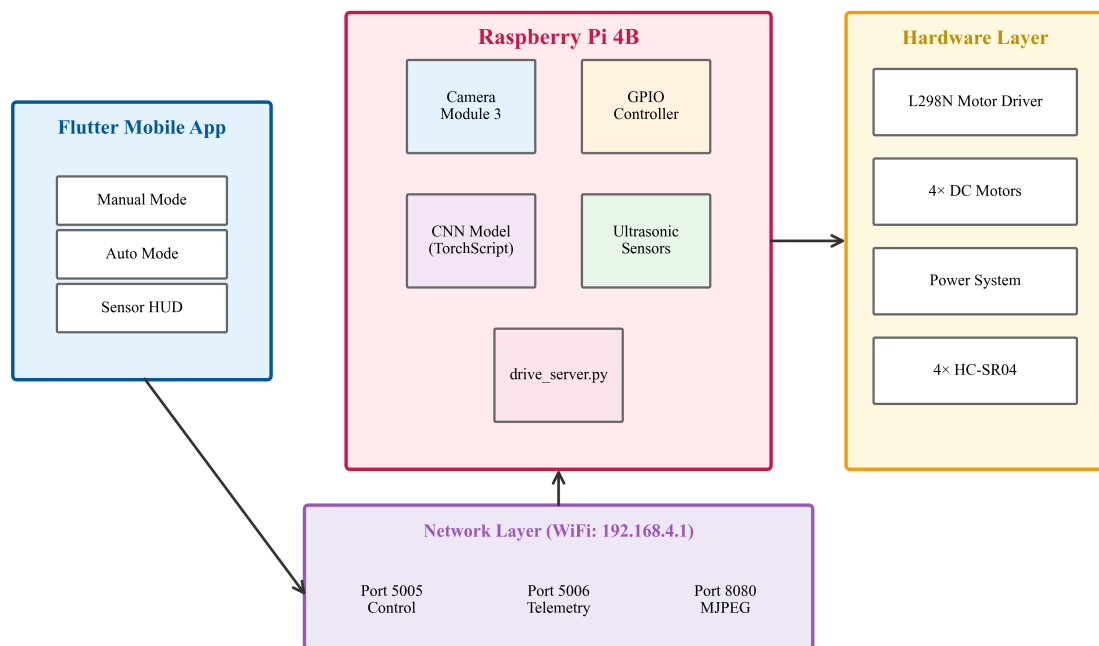
Циљеви овог дипломског рада дефинисани су на следећи начин:

1. Пројектовати и конструисати функционалну платформу аутономног возила са интегрисаним безбедносним механизмима.
2. Имплементирати end-to-end framework за учење засновано на CNN-овима за управљање воланом.
3. Интегрисати ултразвучне сензоре близине за избегавање колизија и оперативну безбедност.
4. Оптимизовати трениране моделе за инференцију у реалном времену на Raspberry Pi хардверу.
5. Развити мобилни контролни интерфејс који пружа визуализацију сензорских података у реалном времену.
6. Евалуирати перформансе система у реалним навигационим сценаријима који укључују избегавање препрека.

1.4 Преглед система

Имплементирани систем прати pipeline илустрован на Слици 1:

1. Прикупљање података: Прикупљање демонстрација људске вожње коришћењем мобилног интерфејса са два joystick-а.
2. Тренирање модела: Тренинг CNN-а на NVIDIA GPU коришћењем PyTorch framework-а са аугментацијом података.
3. Оптимизација модела: Динамичка квантизација за уградбено распоређивање. Постижући убрзање од приближно 1,2x.
4. Примена: Инференција у реалном времену на Raspberry Pi 4 коришћењем улаза камере уживо.
5. Безбедносни слој: Интеграција ултразвучних сензора за детекцију колизија и хитни одговор.
6. Кориснички интерфејс: Мобилна апликација која пружа streaming видеа уживо и визуелизацију сензора путем heads-up display-а (HUD).



Слика 1. Архитектура система високог нивоа која илуструје ток података између хардверских компоненти, софтверских модула и комуникационих слојева.

1.5 Доприноси

Примарни доприноси овог дипломског рада сумирани су на следећи начин:

- Развој комплексног pipeline-а за аутономну вожњу од прикупљања података до уградбене primene.
- Интеграција навигације засноване на визији са безбедносним механизмима заснованим на ултразвучним сензорима.
- Практична евалуација техника оптимизације модела за платформе са ограниченим ресурсима.
- Имплементација стратегије behavioral cloning-а само за управљање воланом, погодне за ограничене скупове података.
- Дизајн мобилног интерфејса за надгледање у реалном времену са визуализацијом сензора.
- Конструкција свеобухватне безбедносне архитектуре са хитним избегавањем препрека.
- Емпиријска анализа behavioral cloning-а и фузије сензора у мобилним роботским системима.

1.6 Организација рада

Остатак овог дипломског рада организован је на следећи начин:

- Поглавље 2 - прегледа сродне радове и постојеће приступе аутономној вожњи.
- Поглавље 3 - описује архитектуру система и дизајнерске одлуке.
- Поглавље 4 - представља методологију учења и процес развоја модела.
- Поглавље 5 - детаљно описује имплементацију софтверских и хардверских компоненти.
- Поглавље 6 - евалуира експерименталне резултате и перформансе система.
- Поглавље 7 - дискутује уочена ограничења и дизајнерске компромисе.
- Поглавље 8 - закључује рад и даје смернице за будући рад.

2. Сродни радови

2.1 Приступи аутономној вожњи

2.1.1 Методе модуларног pipeline-а

Конвенционални системи за аутономну вожњу обично су структурирани као модуларни pipeline-ови који обухватају компоненте за перцепцију, планирање и управљање. Технике као што су симултана локализација и мапирање (SLAM) и планирање путање доминирају овом парадигмом [1]. Иако ефикасни, ови приступи се у великој мери ослањају на ручно пројектовање карактеристике и захтевају комплексну интеграцију подсистема, повећавајући сложеност развоја и трошкове одржавања.

2.1.2 End-to-End учења

End-to-end учење директно мапира сензорске улазе на управљачке излазе, елиминишући потребу међупрезентацијама. Значајан допринос је Nvidia-ина PilotNet архитектура, представљена у раду “End to End Learning for Self-Driving Cars” [2], која је демонстрирала CNN-базирано предвиђање управљања воланом из сирових слика камере. Ова парадигма смањује сложеност система и омогућава имплицитно учење карактеристика, дозвољавајући мрежи да аутономно открије релевантне визуелне обрасце.

2.2 Behavioral Cloning и имитационо учење

Behavioral cloning учи политике управљања из експертских демонстрација кроз надгледано учење [3]. Упркос својој једноставности, овај приступ прати од померања дистрибуције скупа података и акумулације грешке током примене. Уобичајне стратегије ублажавања укључују аугментацију података, прикупљање података за опоравак и итеративне методе агрегације скупа података као што је DAgger [4].

2.3 CNN архитектура за визуелну перцепцију

Класичне CNN архитектуре за класификацију слика укључују AlexNet [5], VGG [6] и ResNet [7], док су лагане архитектуре као што су MobileNet [8] и SqueezeNet [9] специфично оптимизоване за уградбену примену. Овај рад користи прилагођену CNN архитектуру инспирисану NVIDIA-иним PilotNet-ом, балансирајући тачност предикције и рачунарске трошкове.

2.4 Уградбена вештачка интелигенција и Edge Computing

Edge инференција смањује латенцију, побољшава приватност и елиминише зависност од мрежне повезаности [10]. Технике оптимизације модела примењиве на уградбено распоређивање укључују квантизацију, резивање и дестилацију знања [11]. Raspberry Pi платформа нуди атрактиван баланс приступачности и могућности, иако намеће строга ограничења перформанси која захтевају пажљив дизајн модела.

2.5 Сродне платформе

Релативне платформе за аутономна возила укључују Donkey Car пројекат [12], AWS DeepRacer [13] и различита академска истраживачка возила. Међутим, многе постојеће платформе захтевају скупе хардверске компоненте или се ослањају на cloud рачунарство, мотивишући развој потпуно локалне, јефтине алтернативе представљене у овом раду.

2.6 Истраживачки јаз

Овај пројекат адресира ограничења постојећих система омогућавајући потпуно аутономну навигацију засновану на визији на јефтином уградбеном хардверу са интегрисаним безбедносним механизмима који раде независно од научене политике управљања.

3. Архитектура и дизајн система

3.1 Архитектонски преглед

Систем за аутономну навигацију предложен у овом дипломском раду дизајниран је као дистрибуирана сајбер-физичка платформа која комбинује уградбено рачунарство, перцепцију у реалном времену, бежичну комуникацију и мобилну корисничку интеракцију. Систем омогућава аутономну вожњу засновану на визији уз одржавање оперативне безбедности кроз независно сензирање близине.

На високом нивоу, систем са састоји од четири повезана подсистема:

1. Уградбена платформа возила
2. Pipeline за тренинг и оптимизацију модела
3. Мобилна апликација за контролу и надгледање
4. Слој бежичне комуникације и streaming-а података

Укупан дизајн прати модуларну архитектуру која омогућава независан развој, тестирање и замену појединачних компоненти уз очување функционалности на нивоу система.

3.2 Хардверска архитектура

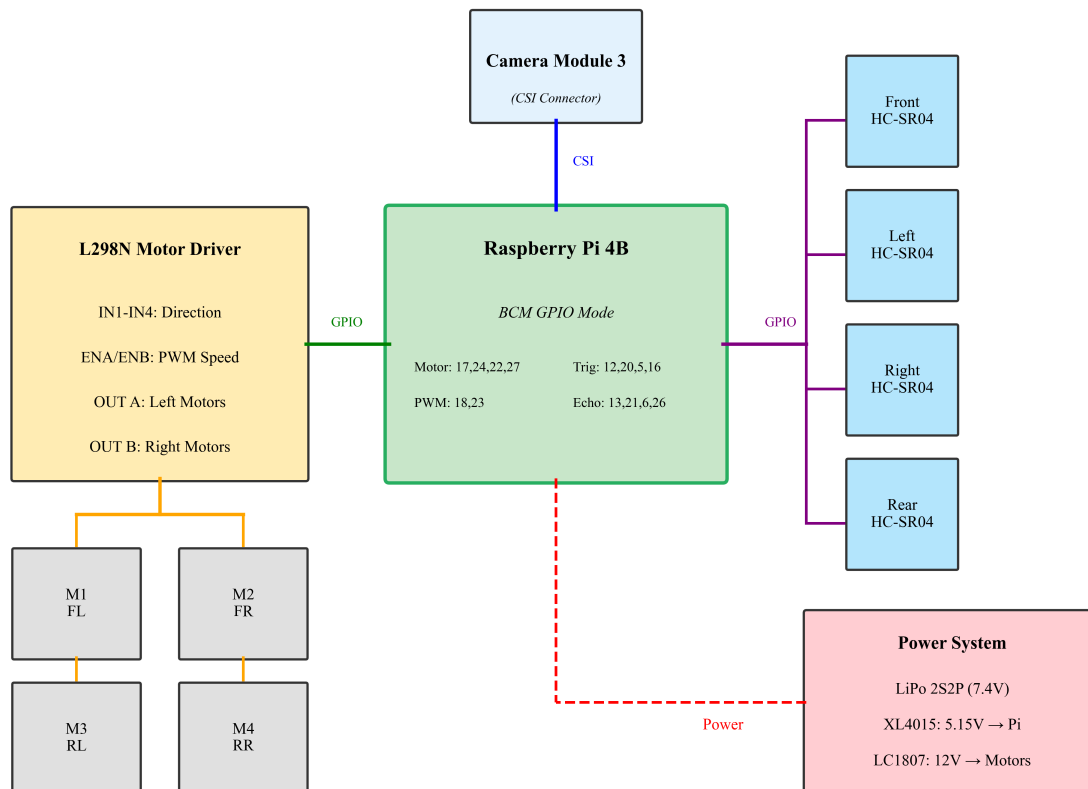
3.2.1 Уградбена рачунарска јединица

Централна рачунарска јединица је Raspberry Pi 4 модел Б опремљен са 4 GB RAM-а. Ова платформа је изабрана због повољног баланса између рачунарских могућности, цене, потрошње енергије и подршке заједнице.

Кључне спецификације укључују:

- Четворојезгарни ARM Cortex-A72 процесор на 1,5 GHz
- 4 GB LPDDR4 RAM
- Camera Serial Interface (CSI) за директну комуникацију камере
- Интегрисани IEEE 802.11ac Wi-Fi и Bluetooth 5.0

Raspberry Pi извршава све задатке у реалном времену, укључујући аквизицију сензора, инференцију неуронске мреже, управљање моторима и мрежну комуникацију.

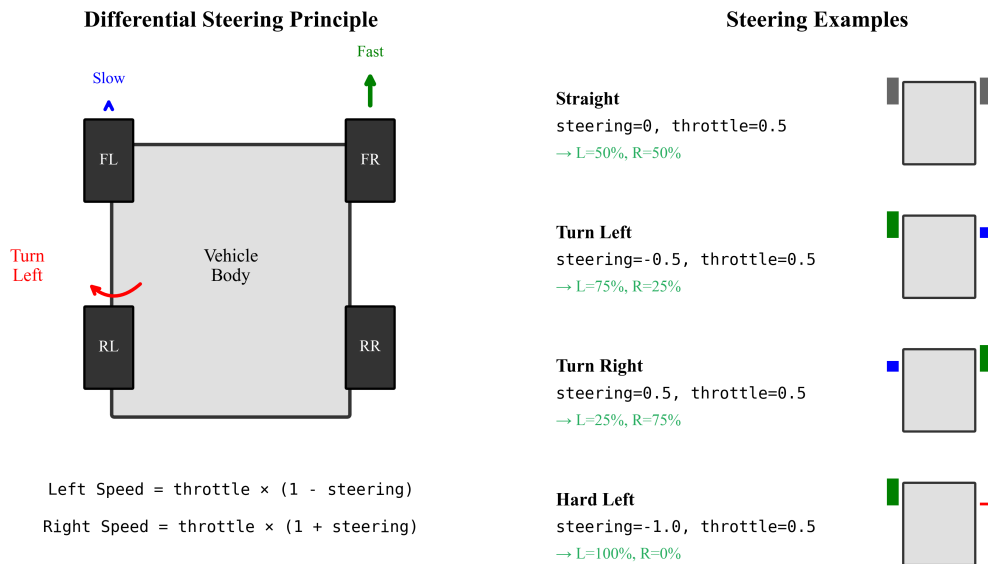


Слика 2. Дијаграм повезивања хардвера који приказује Raspberry Pi 4, драјвер мотора, модул камере и ултразвучне сензоре.

3.2.2 Актуаторски систем

Кретање возила се постиже коришћењем четири DC мотора погођена L298N дуплим X-мост контролером мотора. Диференцијално управљање је имплементирано независном контролом брзина левих и десних мотора.

Pulse-width modulation (PWM) сигнали генерисани путем GPIO пинова регулишу брзину мотора, док дирекциони контролни пинови одређују кретање напред и назад. Ова конфигурација омогућава глатко, континуирано управљање воланом засновано на предикцијама модела.



Слика 3. Механизам диференцијалног управљања који илуструје односе брзина мотора за различите управљачке команде.

3.2.3 Сензорски подсистем

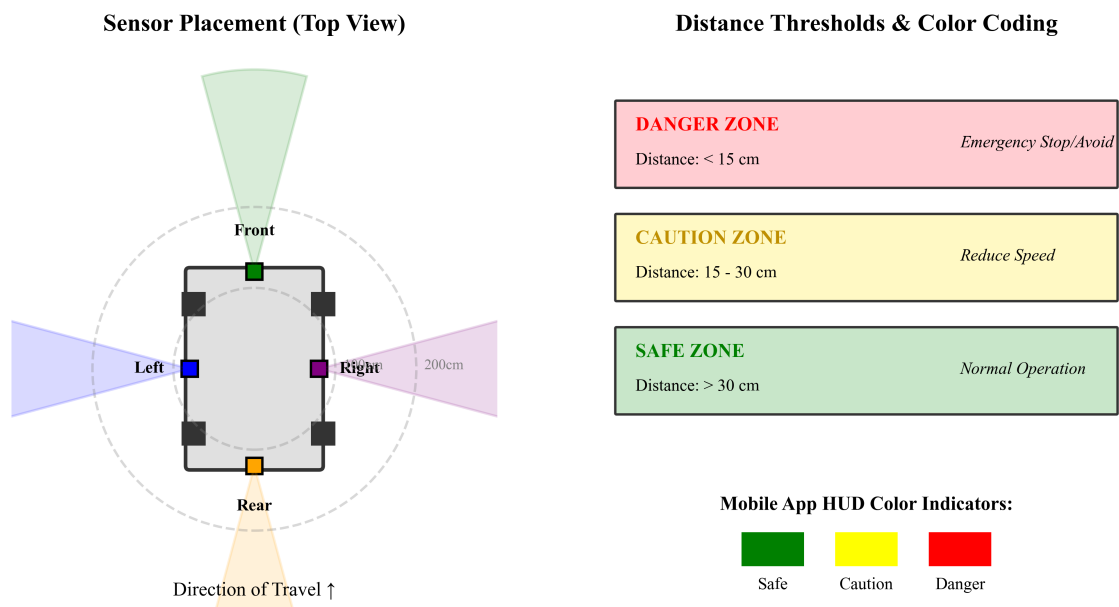
Визуелни сензор: Визуелна перцепција је обезбеђења Raspberry Pi Camera Module 3, базираном на Sony IMX708 сензору слике. Камера испоручује RGB frame-ове који се користе као једини улаз у модел неуронске мреже.

Користе се две конфигурације резолуције:

- 640x480 пиксела за streaming видеа уживо ка мобилној апликацији
- 224x224 пиксела за инференцију неуронске мреже

Ова раздвајања минимизују рачунарски overhead уз очување употребљивости за надгледање и debugging.

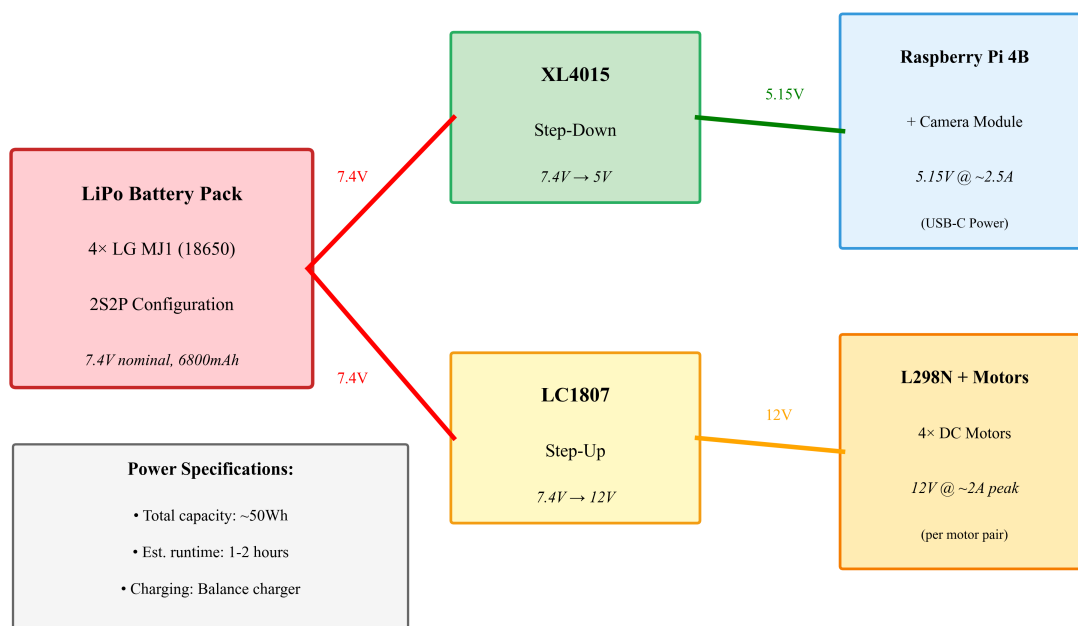
Сензори близине: За осигурање оперативне безбедности, четири HC-SR04 ултразвучна сензора дистанце монтирани су на предњој, левој, десној и задњој страни возила. Сваки сензор мери дистанцу у опсегу од приближно 2-400 cm и ради независно од pipeline-а за визију. Ови сензори пружају редундантни безбедносни слој способан да детектује препреке које камере не може поуздано видети.



Слика 4. Дијаграм распореда ултразвучних сензора који приказује зоне покривености и домете детекције.

3.2.4 Систем дистрибуције напајања

Платформа се напаја пуњивим литијум батеријским пакетом који обезбеђује регулисан напон за Raspberry Pi (5V), драјвер мотора (12V) и сензоре. Електрична изолација између логичког и моторског нивоа напајања смањује електромагнетне сметње и побољшава стабилност система.



Слика 5. Архитектура система напајања која приказује регулацију напона и путању дистрибуције.

3.2.5 Конфигурација GPIO пинова

Табела 1 сумира доделе GPIO пинова за контролу мотора и ултразвучне сензоре.

Функција	BCM Пин	Конекција
Мотор IN1	17	L298N IN1
Мотор IN1	24	L298N IN2
Мотор IN1	22	L298N IN3
Мотор IN1	27	L298N IN4
PWM ENA	18	L298N ENA
PWM ENB	23	L298N ENB
Предњи Trigger	12	HC-SR04 Предњи
Предњи Echo	13	HC-SR04 Предњи
Леви Trigger	20	HC-SR04 Леви
Леви Echo	21	HC-SR04 Леви
Десни Trigger	5	HC-SR04 Десни
Десни Echo	6	HC-SR04 Десни
Задњи Trigger	16	HC-SR04 Задњи
Задњи Echo	26	HC-SR04 Задњи

Табела 1. Конфигурација GPIO пинова

Motor Control Pins

Function	BCM Pin	Connection
Motor IN1	17	L298N IN1
Motor IN2	24	L298N IN2
Motor IN3	22	L298N IN3
Motor IN4	27	L298N IN4
PWM ENA	18	L298N ENA
PWM ENB	23	L298N ENB

Ultrasonic Sensor Pins

Function	BCM Pin	Connection
Front Trig	12	HC-SR04 Front
Front Echo	13	HC-SR04 Front
Left Trig	20	HC-SR04 Left
Left Echo	21	HC-SR04 Left
Right Trig	5	HC-SR04 Right
Right Echo	6	HC-SR04 Right
Rear Trig	16	HC-SR04 Rear
Rear Echo	26	HC-SR04 Rear

Слика 6. Табеле конфигурације GPIO пинова за контролу мотора и ултразвучне сензоре.

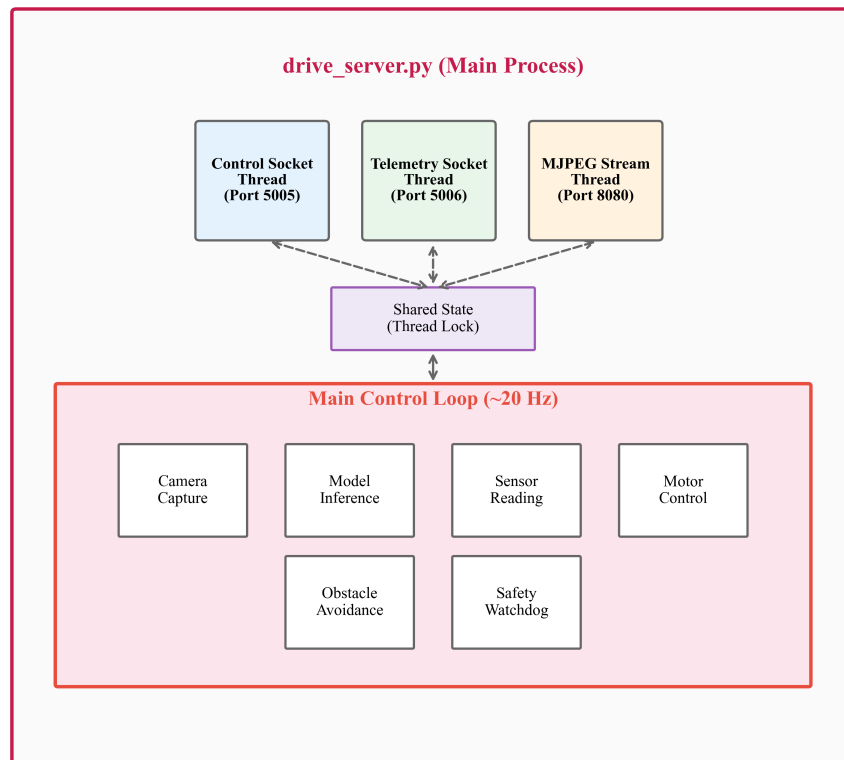
3.3 Софтверска архитектура

3.3.1 Архитектонски преглед

Софтверски систем прати слојевиту и модуларну архитектуру која се састоји од четири главна слоја:

- Уградбени контролни слој
- Слој перцепције и инференције
- Слој корисничке интеракције
- Слој комуникације и телеметрије

Сваки слој излаже добро дефинисане интерфејсе и комуницира користећи структуриране формате података (JSON).



Слика 7. Дијаграм серверске архитектуре који приказује вишенитни дизајн и ток података.

3.3.2 Уградбени контролни слој

Уградбени контролни слој се извршава директно на Raspberry Pi-ју и управља операцијама у реалном времену. Његове примарне одговорности укључују:

- Активацију мотора путем PWM контроле
- Аквизицију сензорских података
- Безбедносно надгледање и watchdog надзор
- Извршавање главне контролне петље

Контролна петља ради на приближно 20 Hz, представљајући баланс између одзива система и рачунарског оптерећења.

3.3.3 Слој перцепције и инференције

Овај слој обрађује frame-ове камере и извршава инференцију неуронске мреже. Тренирани CNN модел, експортиран у TorchScript формату, предвиђа управљачке команде из препроцесираних RGB slika.

Инференција се извршава синхроно унутар контролне петље, осигуравајући детерминистично понашање и конзистентне карактеристике латенције.

3.3.4 Безбедносни слој и руковање грешкама

Безбедносни механизми су имплементирани као паралелни подсистем независан од научене политике вожње. Овај подсистем укључује:

- Ултразвучну детекцију препрека
- Логику хитног кочења
- Извршавање маневара за опоравак
- Watchdog-базирану детекцију timeout-а комуникације

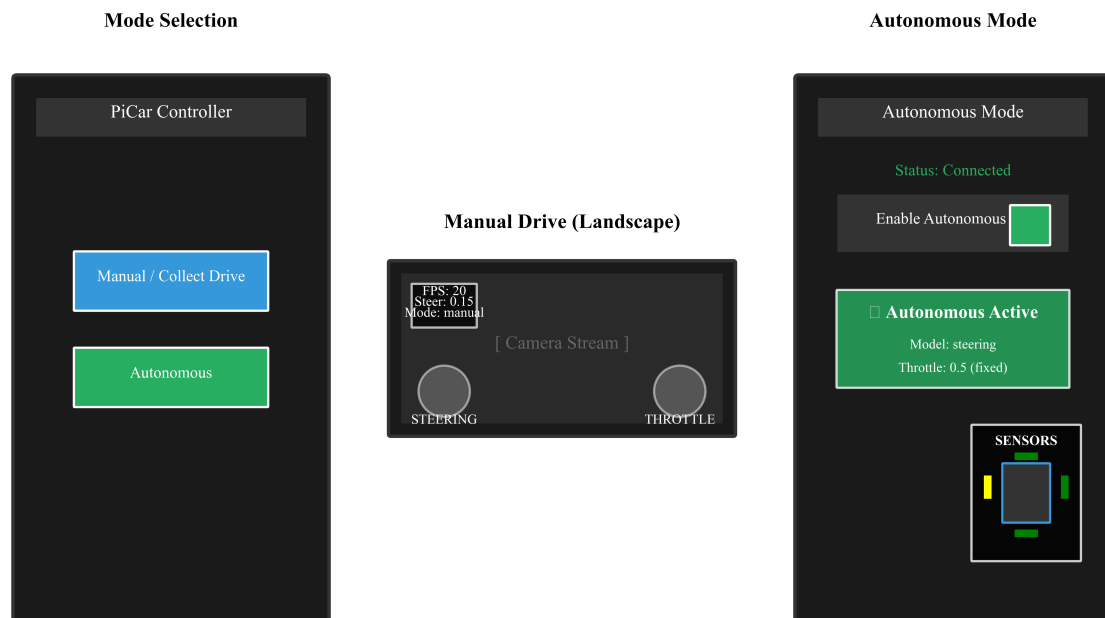
Овај дизајн осигурава да безбедносно критичне одлуке нису зависне од излаза неуронске мреже, чиме се побољшава укупна поузданост система.

3.3.5 Слој мобилне апликације

Мобилна апликација развијена коришћењем Flutter framework-а пружа:

- Ручну контролу вожње путем дуалних виртуелних joystick-ова
- Визуализацију streaming-а камере уживо
- Приказ телеметрије у реалном времену са сензорским HUD-ом
- Активацију аутономног режима и ручно преузимање контроле

Апликација функционише и као даљински контролер и као интерфејс за надгледање током експерименталне евалуације.



Слика 8. Wireframe-ови Flutter мобилне апликације који приказују интерфејсе за ручну контролу и аутономни режим.

3.4 Комуникациона архитектура

3.4.1 Мрежна топологија

Raspberry Pi ради као посвећени Wi-Fi access point користећи статичку IP адресу (192.168.4.1). Мобилна апликација се повезује директно на ову мрежу, формирајући затворени локални систем са предвидивом латенцијом и без екстерних зависности.

Ова топологија осигурава:

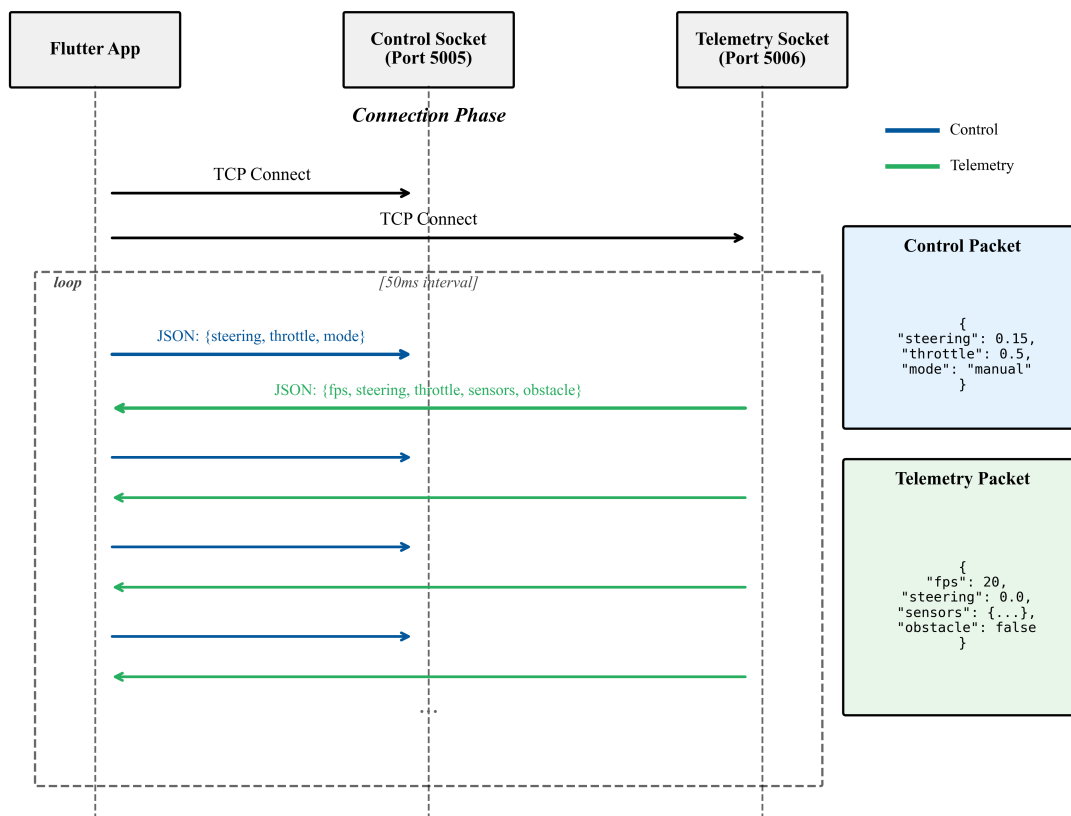
- Ниску и конзистентну комуникациону латенцију
- Високу поузданост без зависности интернета
- Поједностављену мрежну конфигурацију

3.4.2 Комуникациони канали

Користе се три независна комуникациона канала:

1. **Контролни (TCP, Port 5005):** Прима команде за управљање воланом и гасом од мобилне апликације као JSON-форматиране поруке.
2. **Телеметријски (TCP, Port 5006):** Преноси читавање сензора, статус система и оперативни режим ка мобилној апликацији.
3. **Канал за streaming видеа (HTTP MJPEG, Port 8080):** Пружа пренос са камере уживо коришћењем JPEG streaming-a.

Раздвајање канала спречава интерференцију између видео података високог пропусног опсега и временски критичних контролних сигнала.



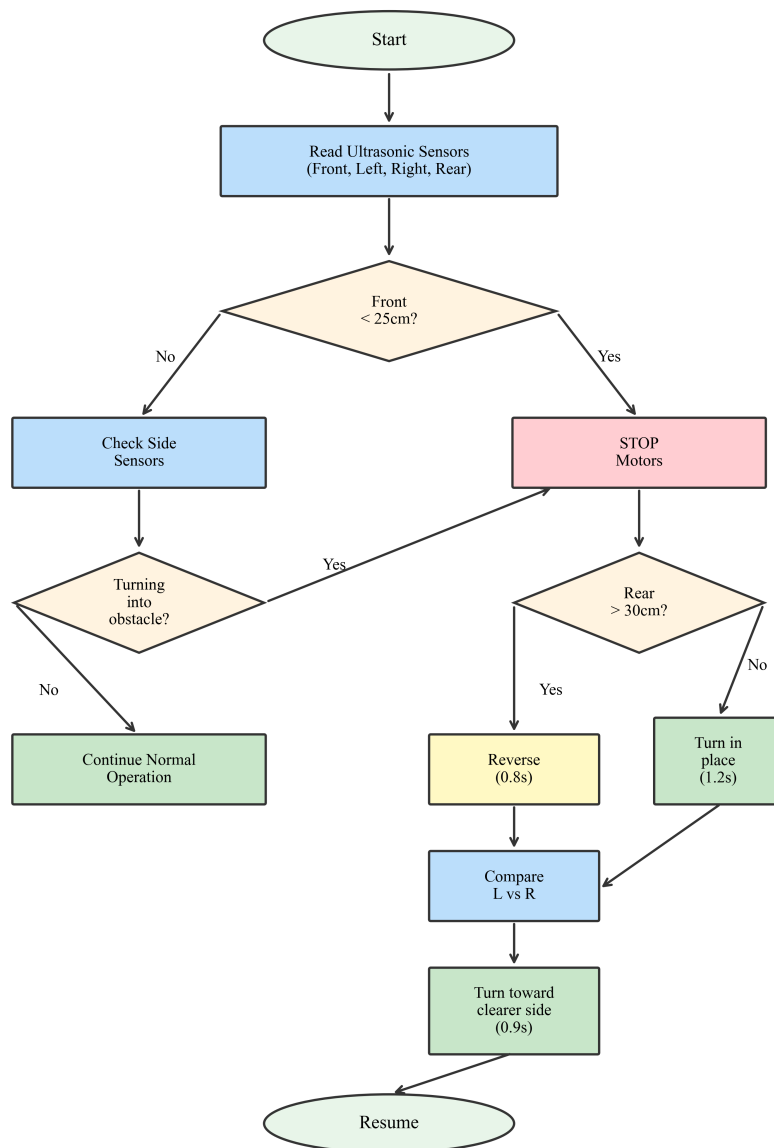
Слика 9. Секвенцни дијаграм комуникације који приказује ток порука између Flutter апликације и серверских компоненти.

3.5 Безбедносна архитектура

Безбедност се третира као првостепени дизајнерски захтев у целом систему. Кључни безбедносни механизми укључују:

- **Watchdog тајмер:** Прекида рад мотора након 500 ms без пријема контролних команди.
- **Ограничење брзине:** Софтверски наметнута максимална ограничења брзине.
- **Детекцију препрека:** Тренутно гашење мотора при детекцији препрека унутар критичних прагова близине.
- **Опоравак свестан смера:** Аутоматизовани маневри вожње уназад и окретања засновани на доступном простору.
- **Грაციозна деградација:** Сигурне транзиције стања при отказу сензора или комуникације.

Безбедносни подсистем одржава приоритет извршења над наученим контролним излазима у сваком тренутку.



Слика 10. Дијаграм тока избегавања препрека који приказује машину стања за хитни одговор.

3.6 Образложење дизајна

Архитектура система одржава неколико фундаменталних дизајнерских принципа:

- **Модуларност:** Омогућава независан развој, тестирање и замену компоненти.
- **Редунданција:** Визија и сензирање близине пружају комплементарне могућности детекције препрека.
- **Детерминизам:** Контролна петља са фиксном стопом побољшава предвидивост временских карактеристика.
- **Fail-Safe рад:** Систем прелази у сигурна стања у условима кvara.

Избор behavioral cloning-a, контроле само воланом и Raspberry Pi хардвера представљају намерни компромис између сложености система и оперативне поузданости.

4. Методологија учења и развој модела

4.1 Формулација проблема

Навигација аутономног возила у овом раду формулисана је као проблем надгледаног учења у коме се политика управљања учи из демонстрација људске вожње. Циљ је апроксимирати мапирање од визуелних обсервација до управљачких команди возила.

Нека $x_t \in \mathbb{R}^{H \times W \times 3}$ означава RGB слику снимљену у временском кораку t , и нека $y_t = [s_t, a_t]$ представља одговарајуће контролне сигнале, где s_t означава команду управљања воланом а a_t означава гас. Циљ учења је апроксимирати функцију:

$$f_\theta : x_t \rightarrow y_t$$

Где је f_θ параметризована конволуционом неуронском мрежом са параметрима учења θ . Ова формулација омогућава директно end-to-end учење без експлицитних међурепрезентација као што су детекција трака или планирање путања.

4.2 Framework за Behavioral Cloning

Behavioral cloning је приступ имитационог учења у коме се политика тренира путем надгледаног учења коришћењем експертских демонстрација [3].

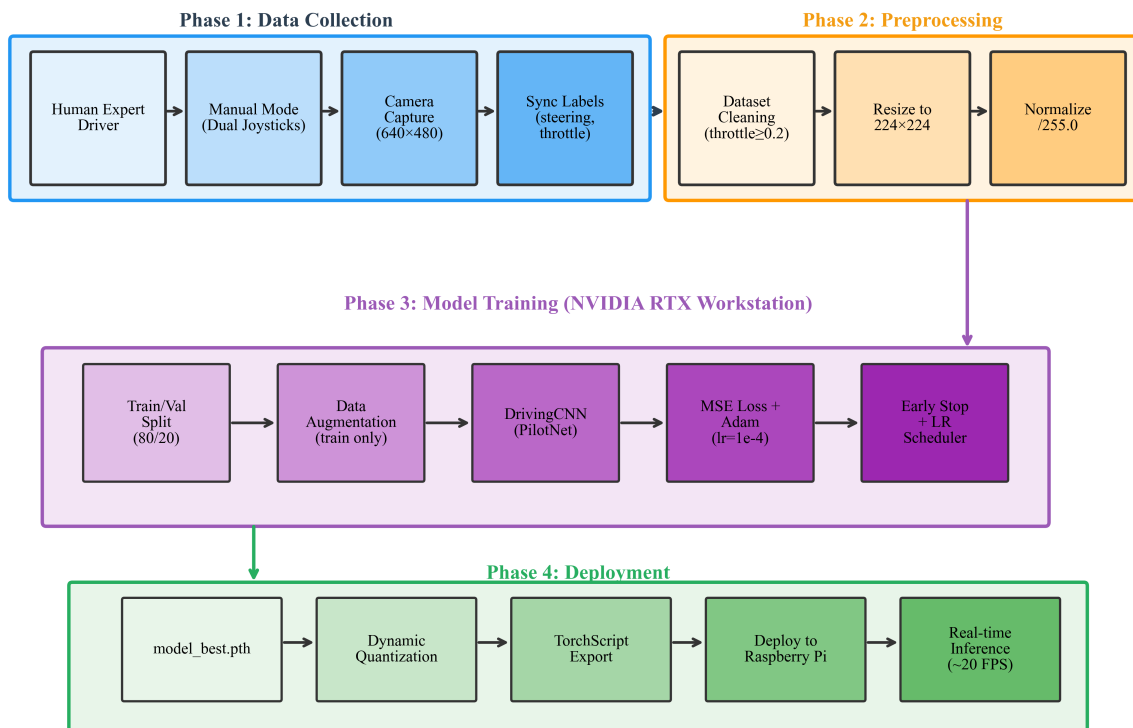
За дати скуп података:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$$

Циљ учења је минимизовати очекивану грешку предикције између излаза модела и контролних сигнала које је дао експерт.

За разлику од појачаног учења (RL), behavioral cloning не захтева симулатор, дизајн функције награде или онлајн истраживање, чинећи га посебно погодним за роботске системе у реалном свету са безбедносним ограничењима.

Међутим, овај приступ је осетљив на пристраност дистрибуције података и акумулацију грешке, што мотивише пажљиве стратегије прикупљања података и аугментације описане у наредним поглављима.



Слика 11. Pipeline за behavioral cloning који приказује фазе прикупљања података, тренинга и примену.

4.3 Репрезентација улаза

Систем користи монокуларне RGB слике као једини перцептуални улаз у неуронску мрежу. Сваки снимљени фрејм пролази кроз следеће трансформације препроцесирања:

1. Просторно скалирање на 224x224 пиксела
2. Конверзија из BGR у RGB простор боја
3. Нормализација интензитета пиксела на опсег $[0, 1]$
4. Стандардизација по каналима коришћењем ImageNet статистика

Ова репрезентација чува довољно просторних и семантичких информација уз остајање рачунарски изводљивом за уградбену инференцију.

4.4 Архитектура модела

4.4.1 Дизајнерска инспирација

Архитектура мреже је инспирисана PilotNet моделом који су предложили Бојарски ет ал. [2], који је демонстрирао ефикасно предвиђање управљања коришћењем конволуционе екстракције карактеристика праћена потпуно повезаних регресионих слојева.

Архитектура је прилагођена да задовољи ограничења уградбеног распоређивања, посебно у погледу латенције инференције и коришћења меморије.

4.4.2 Структура мреже

Архитектура модела, имплементирана у класи DrivingCNN, састоји се од следећих слојева:

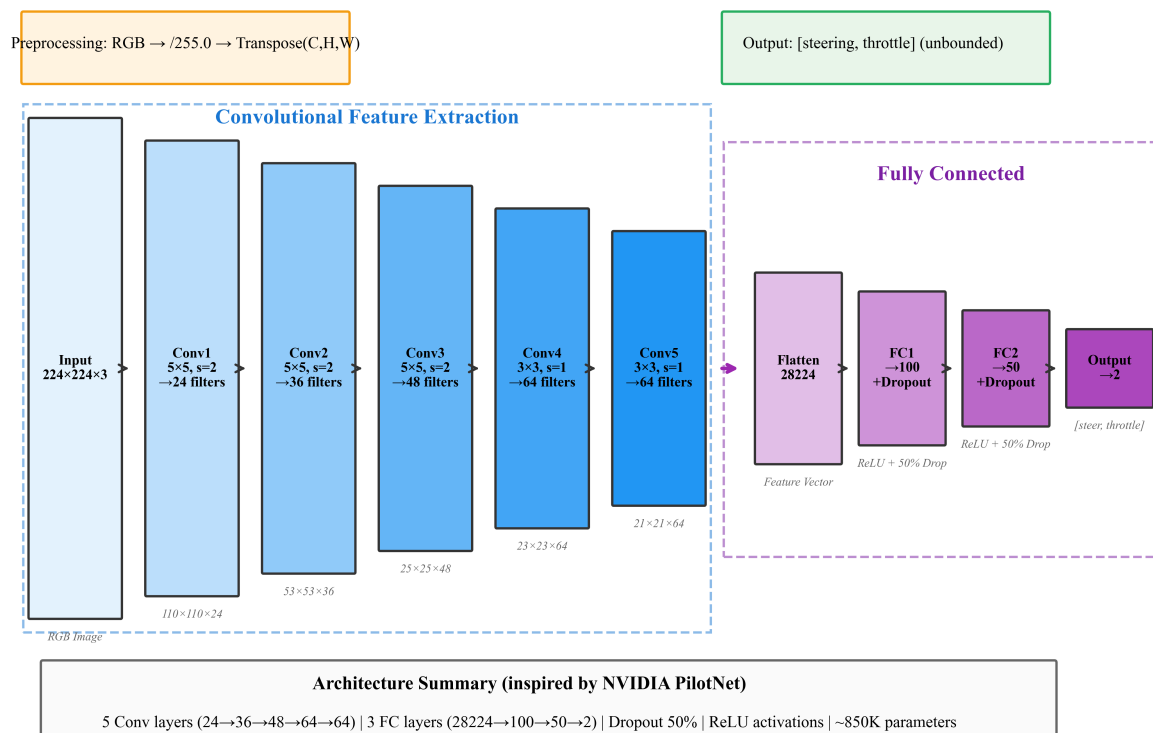
Конволуциона екстракција карактеристика:

- Conv1: 3 → 24 канала, 5x5 kernel, stride 2
- Conv2: 24 → 36, канала, 5x5 kernel, stride 2
- Conv3: 36 → 48 канала, 5x5 kernel, stride 2
- Conv4: 48 → 64 канала, 5x5 kernel, stride 2
- Conv5: 64 → 64 канала, 3x3 kernel, stride 1

Потпуно повезана регресија:

- FC1: Спољашње карактеристике → 100 јединица
- Dropout: $p = 0.5$
- FC2: 100 → 50 јединица
- Dropout: $p = 0.5$
- Излаз: 50 → 2

Rectified Linear Unit (ReLU) активације следе након сваког конволуционог и потпуног повезаног слоја. На излазном слоју се не примењује активациона функција, пошто је задатак формулисан као регресија.



Слика 12. Дијаграм CNN архитектуре који приказује димензије слојева и ток карактеристика.

4.4.3 Динамичко рачунање величине карактеристика

Модел користи динамичко рачунање димензионалности спљоштених карактеристика користећи пролаз унаред са ”лутка” улазним тензором. Овај приступ осигурава компатабилност са различитим улазним резолуцијама без ручног рачунања:

```
def _get_conv_output_dim(self):  
    with torch.no_grad():  
        x = torch.zeros(1, 3, 224, 224)  
        x = self._forward_conv(x)  
        return x.view(1, -1).size(1)
```

4.5 Методологија прикупљања скупа података

4.5.1 Активизација демонстрација

Подаци за тренинг прикупљени су кроз телеоперацију од стране човека коришћењем мобилног контролног интерфејса. Оператер је возио возило ручно док је систем снимао синхронизоване слике и контролне команде на фреквенцији контролне петље (~20 Hz).

Овај процес је произвео лабелиране парове слика-команда погодне за надгледано учење.

4.5.2 Покривеност образаца вожње

За побољшање робусности политике, демонстрације су укључивале:

- Континуалне путање у облику осмице
- Пролазак правим ходницима
- Маневре опоравка близу граница
- Оштре кривине и уске пролазе

Ови обрасци су подстицали изложеност разноврсним угловима управљања и визуелним перспективама.

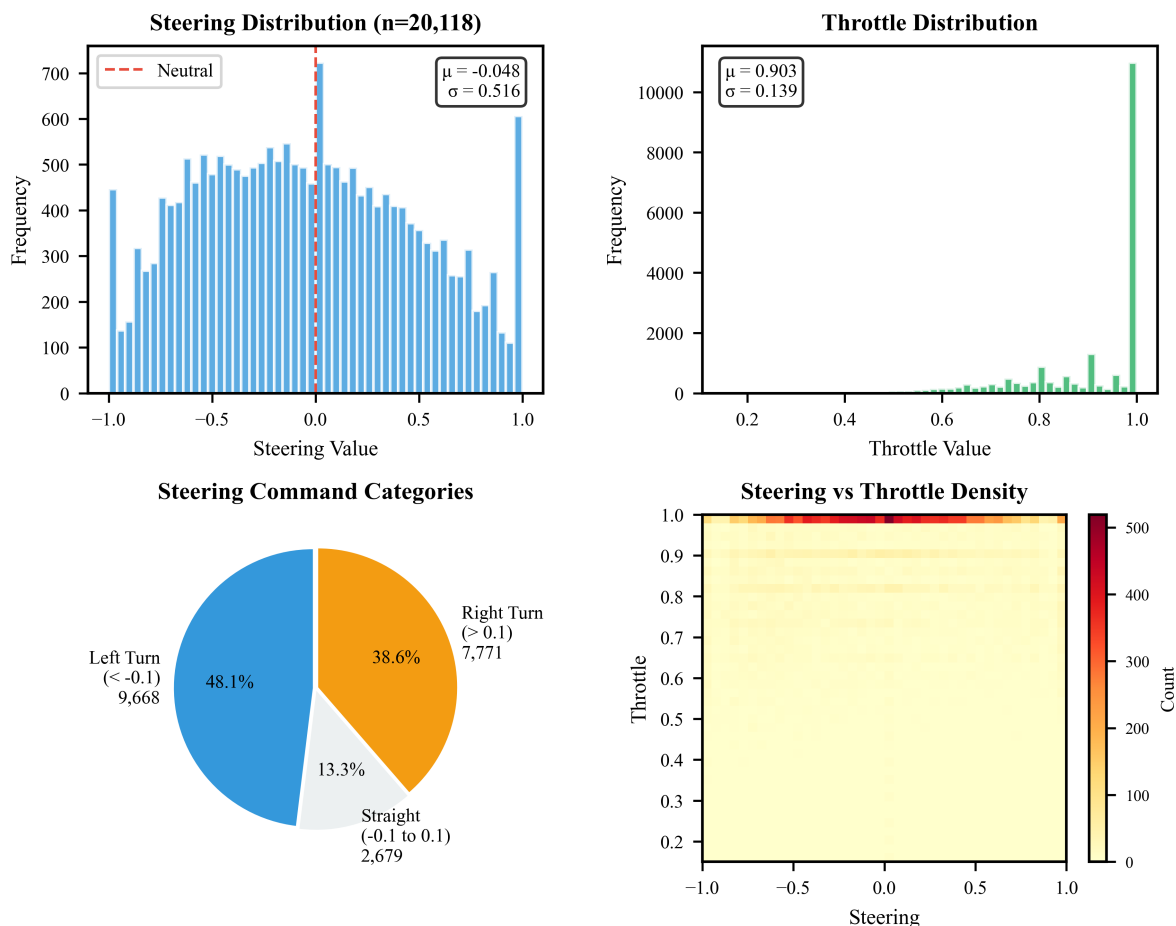
4.5.3 Стратегијско филтрирање података

Да би се спречила пристраност ка стационарном понашању, frame-ови са вредностима гаса испод предефинисаног прага су одбацивани током прикупљања.

Ово филтрирање је осигурало да модел научи динамику вожње уместо стања мировања.

4.5.4 Састав скупа података

Коначни скуп података садржи 20.118 лабелираних узорака, расподељених у скупове за тренинг (80%) и валидацију (20%) коришћењем стратификованог насумичног узорковања са фиксним seed-ом за репродуктивност.



Слика 13. Хистограми дистрибуције података за тренинг који приказују фреквенције вредности управљања воланом и гаса.

4.6 Аугментација података

4.6.1 Pipeline за аугментацију

За побољшање генерализације и ублажавање overfitting-a, стохастичка аугментација се примењује током тренинга коришћењем Albumentations библиотеке:

- **Варијација осветљености/контраста:** Насумично подешавање унутар $\pm 20\%$ ($p = 0.5$)
- **Пертубација нијансе/засићености:** Насумична промена нијансе (± 10), засићеност (± 20), вредности (± 20) ($p = 0.3$)
- Гаусов шум
- Хоризонтално окретање

Аугментација се примењује искључиво на скуп података за тренинг; скуп за валидацију користи само детерминистично препроцесирање.

4.7 Процедура тренинга

4.7.1 Функција губитка

Циљ учења је дефинисан коришћењем Mean Squared Error (MSE) губитка:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \|y_i - \hat{y}_i\|^2$$

Овај губитак третира предикцију управљања воланом и гаса са једнаким тежинама.

4.7.2 Конфигурација оптимизације

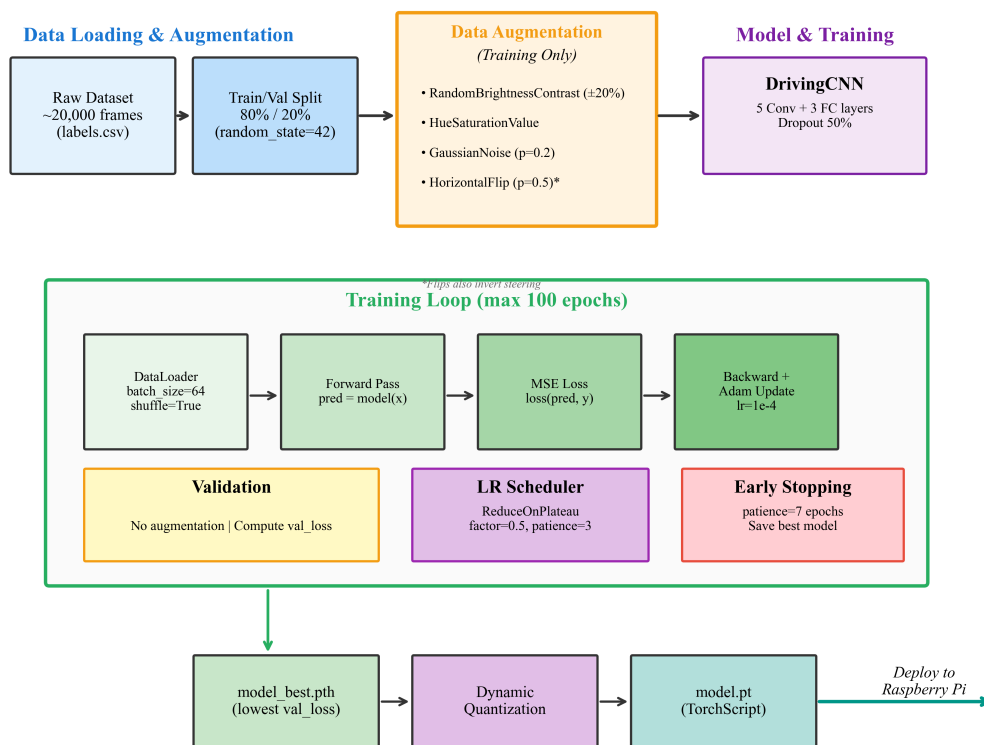
Процедура тренинга користи следећу конфигурацију:

- **Optimizer:** Адам са почетном стопом учења $\alpha = 10^{-4}$
- **Batch Size:** 64 узрока
- **Распоред стопа учења:** ReduceLROnPlateau са фактором 0.5, patience 3 епохе
- **Early Stopping:** Patience од 7 епоха заснован на губитку валидације
- **Максимални број епоха:** 100

4.7.3 Стратегија регуларизације

Више механизма регуларизације је употребљено за побољшање генерализације:

- Dropout ($p = 0.5$) у потпуно повезаним слојевима
- Аугментација података током тренинга
- Early stopping заснован на праћењу губитка валидације



Слика 14. Дијаграм pipeline-а за тренинг који приказује ток података од аугментаице кроз валидацију и експорт модела.

4.8 Оптимизација модела за уградбену примену

4.8.1 Динамичка квантизација

За омогућавање инференције у реалном времену на Raspberry Pi-ју, динамичка post-тренинг квантизација се примењује на линеарне (потпуно повезане) слојеве коришћењем PyTorch-ових алата за квантизацију.

Тежине и активације се конвертују из 32-битне репрезентације са поркетним зарезом (FP32) у 8-битну целобројну репрезентацију (INT8) током инференције, значајно смањујући трошкове и захтеве за пропусним опсегом меморије.

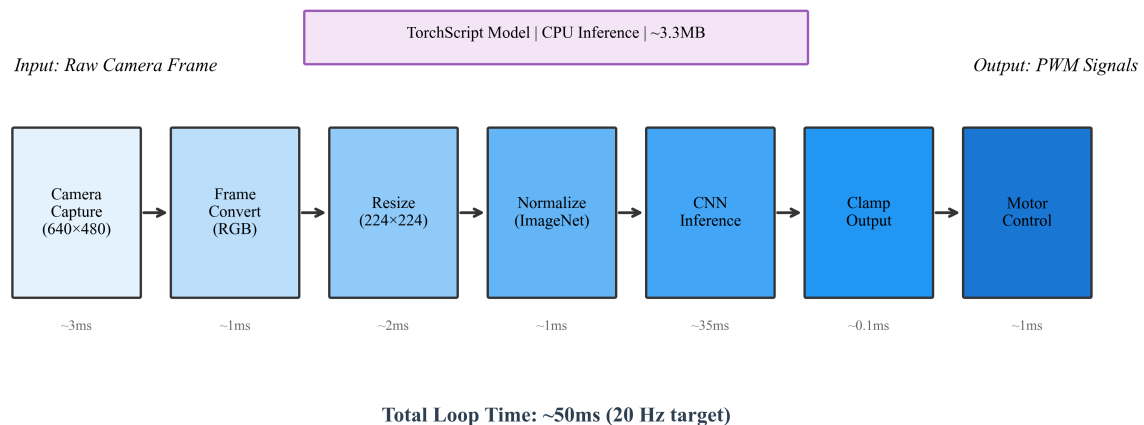
Ова метода постиже убрзање инференције од приближно 1,2x са занемарљивом деградацијом тачности предикције.

4.8.2 TorchScript компилација

Оптимизовани модел се екпортује у TorchScript формат коришћењем компилације засноване на tracing-у:

```
scripted_model = torch.jit.trace(quantized_model, example_input)
scripted_model.save("model.pt")
```

Овим конверзија уклања overhead Python интерпретера и омогућава ефикасно извршење унутар уградбеног runtime-а окружења.



Слика 15. Pipeline инференција у реалном времену који приказује фазе хватања frame-ова, препроцесирања, инференције модела и контроле мотора.

5. Детаљи имплементације

5.1 Преглед

Ово поглавље описује практичну имплементацију система за аутономну навигацију. Док су претходна поглавља увела архитектуру система и методологију учења, ово поглавље се фокусира на софтверску реализацију у више извршених окружења: уградбени Linux систем, pipeline за тренинг на радној станици и мобилна апликација.

5.2 Имплементација прикупљених података

5.2.1 Скрипта за прикупљање

Процес аквизиције података је имплементиран помоћу прилагођене Python скрипте која синхронизује улаз камере и контролне команде током сесија ручне вожње. Скрипта извршава:

- Континуално хватање frame-ова камере резолуције 640x380
- Активизација контролних улаза у реалном времену од мобилне апликације
- Етикетирање података поравнато по временским ознакама
- Онлајн филтрирање неважећих узорака (гас испод прага)
- Трајно складиштење као фајлови слика са CSV-форматираним етикетама

5.2.2 Обрада контролних сигнала

Ручни улаз вожње користи глатку инкременталну контролу. Уместо дискретних промена, контролне вредности се постепено повећавају, производећи континуалне

сигнале управљања који побољшавају глаткоћу етикета и воде ка стабилнијим регресионим циљевима током тренинга.

5.3 Имплементација pipeline-а за тренинг

5.3.1 Dataset класа

Прилагођена класа DrivingDataset имплементира PyTorch Dataset интерфејс:

```
class DrivingDataset(Dataset):
    def __init__(self, csv_file, img_dir, augment=False):
        self.data = pd.read_csv(csv_file)
        self.img_dir = img_dir
        self.augment = augment
        if augment:
            self.aug = A.Compose([
                A.RandomBrightnessContrast(brightness_limit=0.2,
                                           contrast_limit=0.2, p=0.5),
                A.HueSaturationValue(hue_shift_limit=10,
                                     sat_shift_limit=20,
                                     val_shift_limit=20, p=0.3),
                A.GaussNoise(p=0.2),
                A.HorizontalFlip(p=0.5),
            ])
        else:
            self.aug = None
```

Аугментација је условно омогућена само за скуп за тренинг, осигуравајући детерминистичку евалуацију на скупу за валидацију.

5.3.2 Структура скрипте за тренинг

Ток рада тренинга (train_v2.py) обухвата:

1. Иницијализацију скупа података са поделом 80/20 тренинг-валидација
2. Инстанцирање модела са динамичким рачунањем карактеристика
3. Конфигурација optimizer-а (Адам) и scheduler-а (ReduceLROnPlateau)
4. Петљу тренинга са бележењем губитка по епохама
5. Петљу валидације без аугментације
6. Checkpointing најбољег модела заснован на губитку валидације
7. Евалуација early stopping-а

5.4 Имплементација уградбеног сервера

5.4.1 Језгро контролног сервера

Уградбени софтвер (drive_server.py) је имплементиран као централизовани контролни сервер који кординира перцепцију, инференцију, актуацију, комуникацију и безбедносно надгледање. Сервер се аутоматски покреће при старту система путем systemd service конфигурације.

5.4.2 Вишенитна архитектура

За одржавање одзивности, контролни сервер користи вишенитну архитектуру са дедикованим нитима за:

- **Нит контролног socket-a:** Прима JSON-форматиране команде на порту 5005
- **Нит телеметријског socket-a:** Преноси стање система на порту 5006
- **Нит MJPEG streaming-a:** Сервира frame-ове камере на порту 8080
- **Главна контролна петља:** Извршава се на ~ 20 Hz у главној нити

Синхронизација нити користи Python-ов 'threading.Lock' за приступ дељеном стању:

```
state = {  
    "steering": 0.0,  
    "throttle": 0.0,  
    "mode": "manual",  
    "last_cmd": time.time(),  
    "obstacle_detected": False  
}  
lock = threading.Lock()
```

5.4.3 Извршење инференције

TorchScript модел се учитава током иницијализације сервера:

```
model = torch.jit.load("model.pt", map_location=device)  
model.eval()
```

Сваки циклус контроле извршава хватање frame-ова, препроцесирање, инференцију и post-процесирање излаза. Предвиђене вредности управљања се огграничавају на сигурне оперативне опсеге пре актуације мотора.

5.4.4 Стратегије контроле само воланом

Иако модел предвиђа и вредности управљања воланом и гасом, распоређивање користи фиксни гас (0.5) са само предвиђањем управљања воланом од стране модела:

```
if mode == "autonomous":  
    model_s, model_t = get_model_prediction()  
    s = model_s # Model controls steering  
    t = 0.5     # Fixed forward speed  
    set_motors(s, t)
```

Ова одлука је мотивисана уоченом нестабилношћу предикције гаса због небалансираности скупа података, где су стационарни frame-ови доминирали дистрибуцијом етикета гаса.

5.5 Интеграција ултразвучних сензора

5.5.1 Мерење дистанце

Сваки HC-SR04 сензор мери дистанцу засновану на трајању ултразвучног ехо пулса:

```
def get_distance(trig_pin, echo_pin):
    GPIO.output(trig_pin, GPIO.HIGH)
    time.sleep(0.00001)
    GPIO.output(trig_pin, GPIO.LOW)

    # Waiting on echo and measuring the duration of pulse
    pulse_duration = pulse_end - pulse_start
    distance = pulse_duration * 17150 # cm
    return round(distance, 1)
```

Позивање сензора је синхронизовано са контролном петљом да би се осигурало конзистентно временско понашање.

5.5.2 Логика детекције препрека

Прагови детекције су дефинисани засебно за сваки сензор:

- **Предњи сензор:** 25 cm (ризик кретања напред)
- **Бочни сензори:** 15 cm (бочни простор током скретања)
- **Задњи сензори:** 30 cm (простор за вожњу уназад за опоравак)

Логика детекције је активна само током аутономног рада.

5.5.3 Процедура хитног избегавања

При детекцији препреке, систем извршава предефинисану хитну секвенцу:

1. Тренутно заустављање мотора
2. Евалуација простора позади
3. Контролисани маневар вожње уназад (ако има простора)
4. Поређење бочног простора
5. Скретање ка чистијој страни
6. Наставак аутономне навигације

```
def handle_obstacle_avoidance():
    stop()
    time.sleep(0.3)
    distances = get_all_distances()

    if distances['rear'] > 30:
        go_backward(speed=0.6, duration=0.8)
        distances = get_all_distances()

    if distances['left'] > distances['right'] + 15:
        turn_in_place("left", duration=0.9)
    else:
        turn_in_place("right", duration=0.9)
```

5.6 Имплементација мобилне апликације

5.6.1 Framework и архитектура

Мобилна апликација је развијена коришћењем Flutter framework-а са Dart програмским језиком, омогућавајући јединствену базу кода која циља и Android и iOS платформе.

5.6.2 Интерфејс за ручну контролу

Интерфејс за ручну вожњу користи распоред са два joystick-а:

- **Леви joystick:** Контрола управљања воланом (хоризонтална оса)
- **Десни joystick:** Контрола гаса (вертикална оса)

Контролни улази се непрекидно преносе у интервалима од 50 ms (20 Hz) да би се осигурао гладак одзив возила.

5.6.3 HUD за визуализацију сензора

Heads-up display приказује читавања ултразвучних сензора коришћењем репрезентације возила одозго. Дистанце су означене бојама да би се индицирале безбедносне зоне:

- **Зелена:** Сигурно (> 30 cm)
- **Жута:** Упозорење (15-30cm)
- **Црвена:** Критично (< 15 cm)

```
Color _getSensorColor(double distance) {  
    if (distance > 30) return Colors.green;  
    if (distance > 15) return Colors.yellow;  
    return Colors.red;  
}
```

5.6.4 Обрада MJPEG stream-а

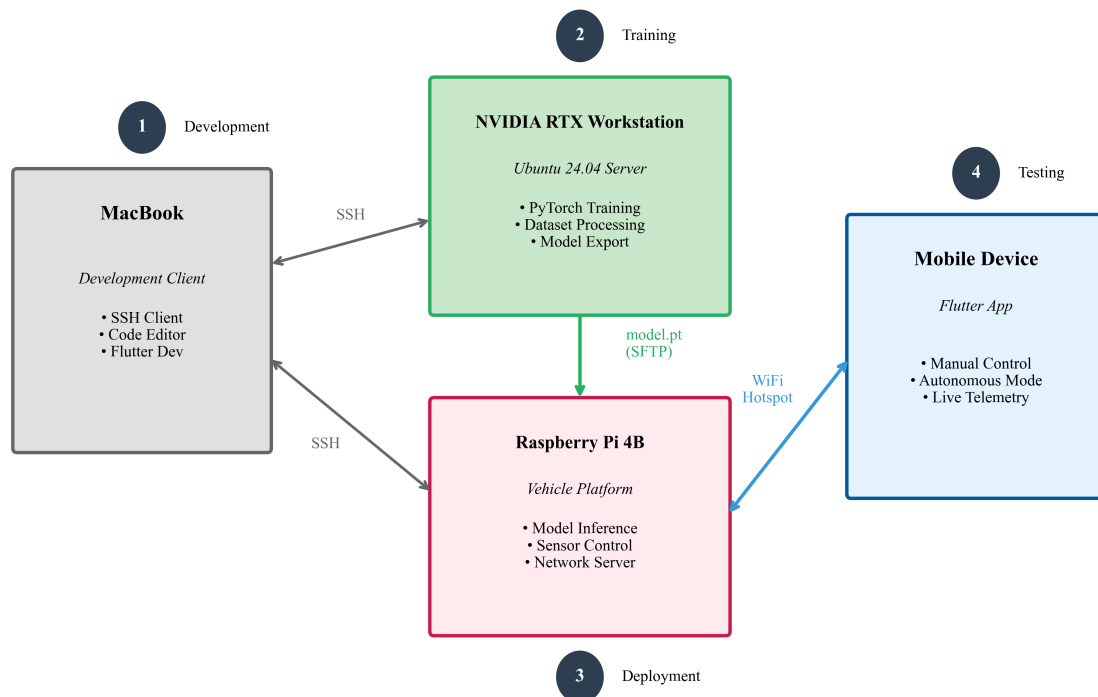
Апликација имплементира прилагођено парсирање MJPEG frame-ова детектовањем JPEG start (0xFF 0xD8) и end (0xFF 0xD9) маркера:

```
final startMarker = [0xFF, 0xD8];  
final endMarker = [0xFF, 0xD9];  
  
// Extract complete JPEG frames from stream buffer  
if (startIndex != -1 && endIndex != -1 && endIndex > startIndex) {  
    final imageData = _imageBytes.sublist(startIndex, endIndex + 1);  
    setState(() => _currentFrame = imageData);  
}
```

5.6.5 Мрежна робусност

Апликација имплементира аутоматско поновно повезивање са експоненцијалним backoff-ом, баферовано JSON парсирање за телеметријске податке и руковање опоравком stream-а за MJPEG конекцију.

5.7 Развојно окружење



Слика 16. Дијаграм развојног окружења који приказује међузвезе радне станице, Raspberry Pi-ја и мобилног уређаја.

6. Експериментална евалуација

6.1 Преглед

Ово поглавље представља експерименталну евалуацију предложеног система за аутономну навигацију, фокусирајући се на перформансе офлајн учења и понашање у раду у реалним условима.

Циљеви евалуације су:

- Анализа карактеристике прикупљеног скупа података за тренинг
- Евалуирати конвергенцију и генерализацију током тренинга модела
- Проценити изводљивост инференције у реалном времену на уградбеном хардверу
- Евалуирати квалитативно понашање вожње у аутономном раду
- Идентификовати ограничења система и случајеве неуспеха

6.2 Експериментална поставка

6.2.1 Хардверско окружење

Сви експерименти су спроведени коришћењем уградбене платформе аутономног возила описане у Поглављу 3. Тренинг модела је извршен на радној станици опремљеној NVIDIA GPU акцелерацијом, док је евалуација инференције спроведена искључиво на Raspberry Pi 4.

6.2.2 Софтверско окружење

Окружење за тренинг је користило:

- Python 3.13.7 са PyTorch framework-ом за дубоко учење
- NumPy за нумеричке операције
- OpenCV за препроцесирање слика
- Albumentations за аугментацију података
- TorchScript runtime за примену

6.3 Анализа скупа података

6.3.1 Дистрибуција управљања воланом

Анализа вредности управљања воланом открива снажну централну тенденцију око нуле, што одговара понашању вожње у правој линији. Ова дистрибуција је типична за скупове података возила и одржава природну доминацију кретања напред током навигације.

Статистичка својства:

- Средња вредност: -0,0477
- Варијанса: 0,2667
- Асиметрија: 0,1709

Средња вредност близу нуле указује на добро центриран скуп података са уравнотеженим понашањем скретања лево и десно. Блага позитивна асиметрија сугерише маргиналну пристраност ка вредностима десног скретања, иако величина остаје мала.

6.3.2 Дистрибуција гаса

Вредност гаса показује значајно мању варијансу:

- Средња вредност: 0,9032
- Варијанса: 0,0195
- Асиметрија: -1,5380

Висока средња вредност и ниска варијанса потврђују ограничену разноликост у контроли гаса, док снажна негативна асиметрија одржава концентрацију близу горњег опсега. Ове карактеристике оправдавају одлуку о распоређивању да се користи фиксни гас.

Метрика	Управљање	Гас
Средња вредност	-0,0477	0,9032
Варијанса	0,2667	0,0195
Асиметрија	0,1709	-1,5380
Минимална	-1,0	0,0
Максимална	1,0	1,0

Табела 2. Статистички преглед скупа података

6.4 Перформансе тренинга

6.4.1 Понашање конвергенције

Криве губитка тренинга и валидације демонстрирају стабилну конвергенцију током сукцесивних епоха. Губитак валидације блиско прати губитак тренинга, указујући на ограничен *overfitting* захваљујући ефикасној регуларизацији кроз аугментацију и *dropout*.

Модел је типично конвергирао унутар 20-30 епоха, након чега је додатни тренинг производио маргинална побољшања. Распоређивање стопе учења је допринело стабилизацији тренинга у касној фази и смањењу осцилација губитка.

Рано заустављање је спречило претренирање и осигурало избор добро генерализованог *checkpoint*-а модела.

6.4.2 Коначне перформансе модела

Коначни модел је постигао ниску средњу квадратну грешку на скупу за валидацију:

- MSE валидације (Управљање): $\sim 0,015$
- MSE валидације (Гас): $\sim 0,008$

6.5 Перформансе уградбене инференције

6.5.1 Латенција инференције

Латенција инференције мерена на Raspberry Pi 4:

- **FP32 модел:** ~45-55 ms по frame-u
- **INT8 квантизовани модел:** ~35-45 ms по frame-u
- **Убрзање:** Приближно 1,2x

Ове перформансе подржавају рад у реалном времену на приближно 20-25 Hz, што одговара фреквенцији контролне петље система.

6.5.2 Искоришћење ресурса

Искоришћење CPU-а је остало унутар прихватљивих граница током аутономног рада, са инференцијом која конзумира већину доступног процесорског времена. Паралелно извршавање мрежних и сензорских нити није значајно деградирало стабилност инференције.

6.6 Евалуација аутономне вожње

6.6.1 Методологија евалуације

Перформансе аутономне вожње су евалуиране кроз поновљене тест вожње у контролисаном унутрашњем окружењу. Свака вожња се састојала од активације аутономног режима праћене континуалном навигацијом без ручне корекције до неуспеха или ручног прекида.

Критеријуми евалуације су укључивали стабилност путање, глаткоћу управљања, понашање опоравка, стопу успешности избегавања препрека и време до неуспеха.

6.6.2 Квалитативна запажања

Током номиналног рада, возило је демонстрирало:

- Глатко понашање праћења траке у тренираним окружењима
- Стабилан излаз управљања са минималном осцилацијом
- Конзистентно кретање напред са фиксним гасом
- Успешно препознавање визуелних образаца за поравнање путања

6.6.3 Перформансе избегавања препрека

Ултразвучни безбедносни подсистем је ефикасно детектовао препреке ван видног поља камере. При детекцији, рутина хитног избегавања је конзистентно извршавала комплетну секвенцу опоравка са високом стопом успешности.

6.6.4 Случајеви неуспеха

Уочено је неколико понављајућих начина неуспеха:

1. **Оштре кривине:** Недовољно података за тренинг за екстремне углове управљања
2. **Транзиција осветљења:** Нагле промене илуминације су изазвале привремене грешке управљања
3. **Површине сиромашне текстуром:** Униформне површине без визуелних карактеристика за навигацију
4. **Замућење услед кретања:** Брзо управљање је производило замућене frame-ове деградирајући квалитет предикције.

Ови неуспеси су конзистентни са познатим ограничењима behavioral cloning-a заснованог на монокуларној визији.

6.7 Анализа генерализације

Систем је демонстрирао умерену способност генерализације унутар окружења визуелно сличних домену тренинга. Перформансе су деградирале у окружењима са различитим текстурама пода, значајно измењеним условима осветљења или невиђеним просторним распоредима.

Ово понашање наглашава снажну зависност имитационог учења од покривености дистрибуције демонстрација.

7. Ограничења и дизајнерски компромиси

7.1 Преглед

Ово поглавље дискутује ограничења уочена током развоја система и експерименталне евалуације, као и кључне дизајнерске компромисе који су утицали на архитектонске и методолошке одлуке.

7.2 Ограничење скупа података

7.3.1 Ограничења монокуларне визије

Систем се ослања искључиво на једну RGB камеру, недостаје му експлицитна перцепција дубине. Сходно томе:

- Процена дистанце је имплицитна уместо експлицитна
- Може доћи до двосмислености размере
- Перформансе деградирају у условима лошег осветљења

7.3.2 Осетљивост на окружење

Научени модел је осетљив на промене текстуре пода, рефлексивности површине, интензитета илуминације и образаца сенки, ограничавајући генерализацију ван домена тренинга.

7.4 Ограничења модела

7.4.1 Одсуство темпоралног моделовања

Неуронска мрежа обрађује сваку слику независно без укључивања темпоралног контекста. Ово ограничење може узроковати закашњење реакције на надолazeће кривине, осцилаторксе корекције управљања и смањену стабилност током брзих маневара.

7.4.2 Реактивна контрола

Научена политика је чисто реактивна и недостају јој способности предиктивног планирања. Иако довољно за навигацију ниском брзином, овај дизајн ограничава перформансе у сложеним сценаријима који захтевају антиципаторно резонување.

7.5 Компромиси уградбеног система

7.5.1 Рачунарска ограничења

Raspberry Pi платформа намеће ограничења на процесорску компоненту, пропусни опсег меморије и капацитет паралелног извршавања. За одржавање перформанси у реалном времену, били су неопходни следећи компромиси:

- Компактна мрежна архитектура са ограниченом дубином
- Квантизација инференција која смањује нумеричку прецизност
- Смањења резолуција слике

7.5.2 Стратегија фиксног гаса

Фиксирање гаса током аутономног рада је смањило нестабилност система и побољшало безбедност по цену елиминисања динамичке адаптације брзине.

7.6 Компромиси безбедносне архитектуре

Хибридни дизајн који комбинује научену контролу са безбедношћу заснованом на правилима уводи дисконтинуитете између подсистема, повремено резултујући наглим транзицијама контроле. Овај компромис је прихваћен с обзиром на приоритизацију физичке безбедности над глаткоћом понашања.

8. Закључак и будући рад

8.1 Закључак

Дипломски рад је представио дизајн, имплементацију и евалуацију end-to-end система за аутономну навигацију заснованог на конволуционим неуронским мрежама и behavioral cloning-у.

Систем је успешно демонстрирао аутономну вожњу у реалном времену на уградбеном хардверу коришћењем монокуларне визије уз одржавање оперативне безбедности кроз независно сензирање близине. Експериментална евалуација је потврдила да се контрола заснована на учењу може практично имплементирати на јефтиним роботским платформама када је подржанма пажљиво пројектованом софтверском инфраструктуром и безбедносним механизмима.

8.2 Остварени циљеви

Примарни циљеви дефинисани на почетку су остварени:

1. Развој комплетног pipeline-а за навигацију вођену подацима од прикупљања до примене
2. Успешан тренинг политике контроле засноване на визији коришћења behavioral cloning-а
3. Примена уградбене инференције у реалном времену са динамичком квантизацијом
4. Интеграција безбедносних механизма и механизма опоравка заснованих на ултразвуку
5. Експериментална валидација у реалним навигационим условима

8.3 Будући рад

Више праваца може бити истражено за проширење и побољшање система:

8.3.1 Темпорални модели учења

Будуће имплементације могу укључити рекурентне неуронске мреже (RNN), темпоралне конволуционе мреже (TCN) или архитектуру засновану на transformers-има за хватање динамике кретања и побољшање стабилности управљања.

8.3.2 Побољшање генерисање скупа података

Потенцијална побољшања укључују демонстрацију више оператера, намерно прокупљање података за опоравак пратећи DAgger парадигму, насумичне домene у симулацији и аутоматизоване технике балансирања података.

8.3.3 Фузија сензора

Комбиновање визије са инерцијалним мерним јединицама (IMU), одометријом точкова или сензорима дубине могло би побољшати робусност и смањивати двосмисленост у изазовним окружењима.

8.3.4 Адаптивна контрола брзине

Поновно увођење научене контроле гаса кроз побољшан дизајн скупа података или засебне лонгтудиналне контролере представљају важно проширење за способнију навигацију.

8.3.5 Тренинг заснован на симулацији

Укључивање симулационих окружења (нпр. CARLA, Gazebo) би омогућило скалабилно генерисање података и сигурније истраживање сложених сценарија пре примене у реалном свету.

8.4 Завршне напомене

Дипломски рад демонстрира да се напредне способности аутономне навигације могу постићи коришћењем јефтиног, широко доступног хардвера кроз пажљив дизајн архитектуре система, модерне методологије дубоког учења и намерну интеграцију комплементатних сензорских модалитета.

Комбинација end-to-end учења са ултразвучним безбедносним механизмима се показала ефикасном за структурирана унутрашња окружења и пружа поуздану основу за будућа истраживачка проширења. Како edge рачунарство технологије настављају да напредују, очекује се да системи ове природе постају све способнији и широко применљиви за едукативне и експерименталне примене.

9. Литература

- [1] H. Durrant-Whyte and T. Bailey, "Simultaneous localization and mapping: Part I," **IEEE Robotics & Automation Magazine**, vol. 13, no. 2, pp. 99–110, Jun. 2006.
- [2] M. Bojarski *et al.*, "End to end learning for self-driving cars," **arXiv preprint arXiv:1604.07316**, 2016.
- [3] D. A. Pomerleau, "ALVINN: An autonomous land vehicle in a neural network," in **Advances in Neural Information Processing Systems**, vol. 1, 1989, pp. 305–313.
- [4] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in **Proc. 14th Int. Conf. Artificial Intelligence and Statistics**, 2011, pp. 627–635.
- [5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in **Advances in Neural Information Processing Systems**, vol. 25, 2012, pp. 1097–1105.
- [6] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in **Proc. Int. Conf. Learning Representations**, 2015.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in **Proc. IEEE Conf. Computer Vision and Pattern Recognition**, 2016, pp. 770–778.
- [8] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," **arXiv preprint arXiv:1704.04861**, 2017.
- [9] F. N. Iandola *et al.*, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5 MB model size," **arXiv preprint arXiv:1602.07360**, 2016.
- [10] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," **IEEE Internet of Things Journal**, vol. 3, no. 5, pp. 637–646, Oct. 2016.
- [11] Y. Cheng, D. Wang, P. Zhou, and T. Zhang, "A survey of model compression and acceleration for deep neural networks," **arXiv preprint arXiv:1710.09282**, 2017.
- [12] Donkey Car Project. [Online]. Available: <https://www.donkeycar.com/>
- [13] AWS DeepRacer. [Online]. Available: <https://aws.amazon.com/deepracer/>
- [14] Soleša, D., Arbutina, B.(2024). Uvod u programski jezik C# (p. 99). Fakultet za ekonomiju i inženjerski menadžment.

10. Сумирани преглед слика

Слика	Опис	Назив фајла
Слика 1	Архитектура система високог нивоа	01_system_architecture.tiff
Слика 2	Дијаграм повезивања хардвера	08_hardware_connection.tiff
Слика 3	Механизам диференцијалног управљања	10_differential_steering.tiff
Слика 4	Распоред ултразвучних сензора	11_ultrasonic_sensor_placement.tiff
Слика 5	Архитектура система напајања	14_power_system_architecture.tiff
Слика 6	Конфигурација GPIO Нинова	15_gpio_pin_configuration.tiff
Слика 7	Дијаграм серверске архитектуре	04_server_architecture.tiff
Слика 8	Wireframe-ови Flutter апликације	09_flutter_app_wireframes.tiff
Слика 9	Секвенцни дијаграм комуникације	07_communication_sequence.tiff
Слика 10	Дијаграм тока избегавања препрека	05_obstacle_avoidance_flowchart.tiff
Слика 11	Pipeline за behavioral cloning	03_behavioral_cloning_pipeline.tiff
Слика 12	Дијаграм CNN архитектуре	02_cnn_architecture.tiff
Слика 13	Дистрибуција података за тренинг	06_training_data_distribution.tiff
Слика 14	Дијаграм pipeline-а за тренинг	16_training_pipeline_augmentation.tiff
Слика 15	Pipeline инференција у реалном времену	13_realtime_inference_pipeline.tiff
Слика 16	Развојно окружење	12_development_environment.tiff

11. Прилог: фотографије vozila

